



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

Greedy algorithm

อัลกอริทึมเชิงละโมภ

ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล

nattagit.j@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

เทคนิคตะลุยหาคำตอบ (Brute force technique)

- Brute force คือหลักการตรงไปตรงมาในการหาคำตอบของปัญหา โดยอาศัยความสามารถของเครื่องคอมพิวเตอร์ในการค้นหาคำตอบ กล่าวอีกนัยหนึ่งคือการตะลุยเพื่อสำรวจคำตอบของปัญหาที่เป็นไปได้ เพื่อให้ได้คำตอบที่ต้องการ



ดังนั้นการ brute force จึงเหมาะสำหรับปัญหาที่ยังไม่มี solution และมีขนาดไม่ใหญ่มาก (small input size)

ปัญหาการแลกเหรียญ (Coin changing problem)

- แคชเชียร์ 7-11 มีเหรียญ 1 บาท 2 บาท 5 บาท และ 10 บาท จำนวนไม่จำกัด ต้องการทอนเงิน 57 บาท อยากทราบว่าจะต้องใช้เหรียญเหล่านี้ในการทอนเงินให้พอดีน้อยที่สุดเท่าไร



อัลกอริทึมเชิงละโมภ (Greedy algorithm)

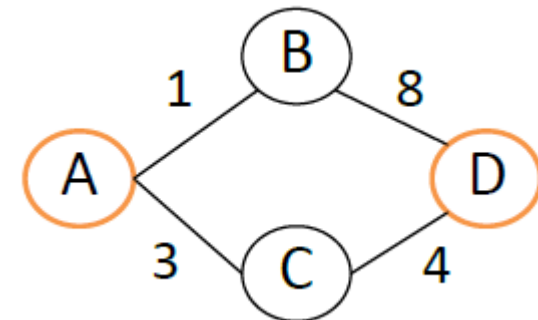
- อัลกอริทึมเชิงละโมภเป็นเทคนิคที่เหมาะสมในการแก้ปัญหาเพื่อค้นหาคำตอบที่เหมาะสมที่สุด (optimal solution)
- หลักการเชิงละโมภ ก็คือการลดเวลาโดยการตัดสินใจเลือกคำตอบย่อย (sub solutions) ที่ดีที่สุดในขณะนั้น โดยที่คำตอบที่เลือกจะต้องไม่ขัดแย้งกับเงื่อนไขบังคับของปัญหา (problem constraint) โดยคาดหวังว่าคำตอบที่เลือกจะได้คำตอบที่เหมาะสมที่สุด
- ดังนั้นประสิทธิภาพของ Greedy จึงจะขึ้นอยู่กับ เกณฑ์ที่นำไปใช้ในการตัดสินใจ (greedy criterion) ในการเลือกแต่ละคำตอบย่อย

เกณฑ์การเลือกของ Greedy

1. ทอนเหรียญที่มีมูลค่ามากที่สุดก่อนให้มากที่สุดเท่าที่จะเป็นไปได้
2. หากเงินมีค่าน้อยกว่ามูลค่าเหรียญแล้ว ให้ทอนเหรียญที่มีมูลค่ารองลงมา
3. ทำซ้ำเหมือนเดิมกับเหรียญที่มีมูลค่าน้อยลง จนกระทั่งเงินทอนมีค่าเป็นศูนย์หรือไม่สามารถทอนเงินได้อีกต่อไป

Greedy Algorithm

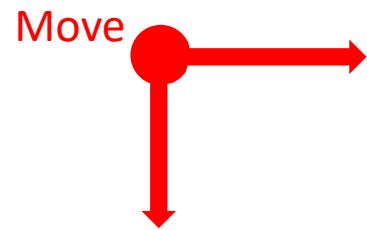
- ข้อดีของอัลกอริทึมเชิงละโมภ
 - การพัฒนาโปรแกรมทำได้ง่าย
 - อัลกอริทึม greedy มักให้เวลาในการทำงานที่ดี
บ่อยครั้งเป็นฟังก์ชัน polynomial
- ข้อเสีย
 - ไม่รับประกันว่าคำตอบที่ได้จะเป็นคำตอบที่ดีที่สุด (optimal solution) เสมอ เนื่องจากการเลือกคำตอบย่อยที่ดีที่สุดทุกครั้งอาจไม่ได้นำไปสู่คำตอบที่ดีที่สุดเสมอไป



ปัญหาเหมาะกับการใช้อัลกอริทึม Greedy

ตัวอย่าง

- ปัญหา knapsack
- ปัญหาการจัดตารางงาน (task scheduling)
- ปัญหาการบีบอัดข้อความ (text compression)



Start	 4	 5	 6
 3	 4	 1	 3
 3	 20	 10	 6
 8	 8	 2	Goal

ปัญหา 0/1 Knapsack

- เลือกของ K ชิ้นใส่กระเป๋าให้มีกำไร (profit) มากที่สุด โดยกระเป๋าไม่ขาด

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8



Weight limit

25

ปัญหา 0/1 Knapsack

คำตอบของปัญหา 0/1 Knapsack สามารถแสดงในรูปของไบนารีเซตโดยที่ไอเทมซึ่งถูกเลือก ('1') และไม่ถูกเลือก ('0')

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8

ตัวอย่างการหยิบ

X	Profit	Weight
{0, 0, 1, 1}	$15 + 8 = 23$	$10 + 5 = 15$

เกณฑ์การตัดสินใจ greedy

สมมติจำนวนของ $N = 4$ และน้ำหนักกระเป๋าที่รับได้คือ $M = 25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า $(p_1, p_2, p_3, p_4) = (25, 24, 15, 8)$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = (18, 15, 10, 5)$

1. เลือกของน้ำหนักน้อยที่สุดใส่ก่อน (Minimize weight)
2. เลือกของที่มีมูลค่ามากที่สุดใส่ก่อน (Maximize Profit)

Greedy solutions

1. เรียงลำดับสิ่งของตามน้ำหนักที่น้อยที่สุดก่อน
o4->o3->o2->o1

$X = \{0, 0, 1, 1\}$ มูลค่ารวม $P = 23$

2. เรียงลำดับสิ่งของตามมูลค่าจากมากไปน้อย
o1->o2->o3->o4

$X = \{1, 0, 0, 1\}$ มูลค่ารวม $P = 33$

Item	Weight	Profit
O1	18	25
O2	15	24
O3	10	15
O4	5	8

M = 25

A good strategy for 0/1 knapsack

- เลือกสิ่งของโดยนำของซึ่งมีมูลค่ามากแต่น้ำหนักน้อยใส่ถุงก่อน
- กำหนดให้ความหนาแน่น (**density**) คือสัดส่วนของมูลค่าและน้ำหนักของวัตถุ ตัวอย่างเช่น วัตถุ A มีมูลค่า 25 น้ำหนัก 18 จะมีความหนาแน่นคือ $25/18 = 1.39$
- หลักการของ Greedy
 - เรียงลำดับของแต่ละชิ้นตามความหนาแน่นจากมากไปน้อย
 - หยิบใส่ตามลำดับที่เรียงใหม่จนกระทั่งไม่สามารถใส่ได้ (ถุงขาด)

	O1	O2	O3	O4
Profit	25	24	15	8
Weight	18	15	10	5
Density	1.39	1.6	1.5	1.6

ตัวอย่าง Greedy สำหรับ **0/1 Knapsack**

สมมติ $n = 4$ และน้ำหนักถุงที่รับได้คือ $M=25$ กำหนดให้ของแต่ละชิ้นมีมูลค่า

$(p_1, p_2, p_3, p_4) = \{25, 24, 15, 8\}$ และน้ำหนักคือ $(w_1, w_2, w_3, w_4) = \{18, 15, 10, 5\}$

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : $o_2 \rightarrow o_4 \rightarrow o_3 \rightarrow o_1$

(x_1, x_2, x_3, x_4)	Profit	Weight
0 1 0 1	$24+8 = 32$	$15+5 = 20$

ปัญหา **Fractional Knapsack**

- ปัญหา 0/1 Knapsack อนุญาตให้ใส่ของลงไปในถุงทั้งวัตถุ ซึ่งมักจะทำให้ถุงเหลือน้ำหนักบางส่วนทิ้งไว้ ตัวอย่างเช่น ในตัวอย่างที่แล้วเราจะเหลือน้ำหนัก 5 หน่วยที่ไม่สามารถใส่ของได้แล้ว
- ปัญหา Fractional knapsack จะเพิ่มเงื่อนไขของปัญหาคือว่าเราสามารถแบ่งของเพื่อนำไปใส่ในถุงได้ แต่มูลค่าของสิ่งนั้นจะลดลงตามสัดส่วนที่แบ่งด้วยเช่นกัน

Greedy strategy for fractional knapsack

วิธีการเลือกตามความหนาแน่นของวัตถุที่ใช้ในปัญหา 0/1 Knapsack จะยังให้คำตอบที่เป็น Optimal หรือไม่?

O1	O2	O3	O4
$25/18 = 1.39$	$24/15 = 1.6$	$15/10 = 1.5$	$8/5 = 1.6$

ใส่ของลงไปในถุงตามลำดับ : o2 -> o4 -> **o3** -> o1

(x1, x2, x3, x4)

0 1 **1/2** 1

Profit

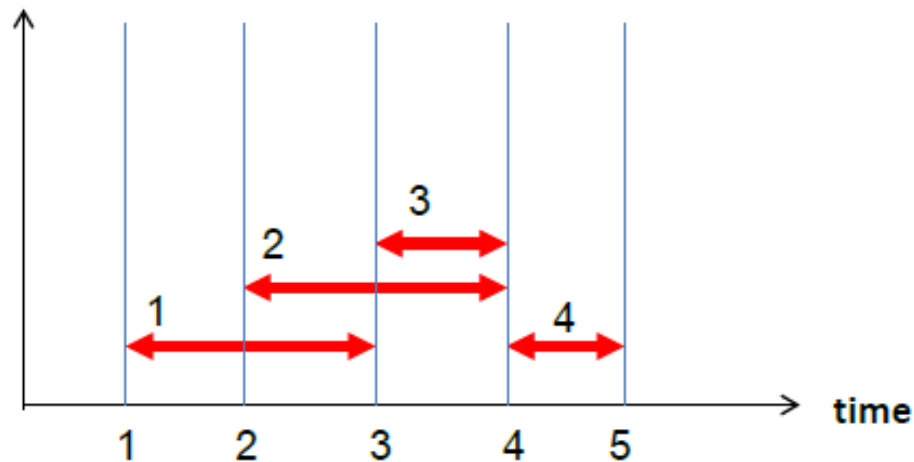
$24 + (15 * \mathbf{1/2}) + 8 = 39.5$

Weight

$15 + 5 + (10 * \mathbf{1/2}) = 25$

ปัญหาการจัดตารางงาน (task scheduling)

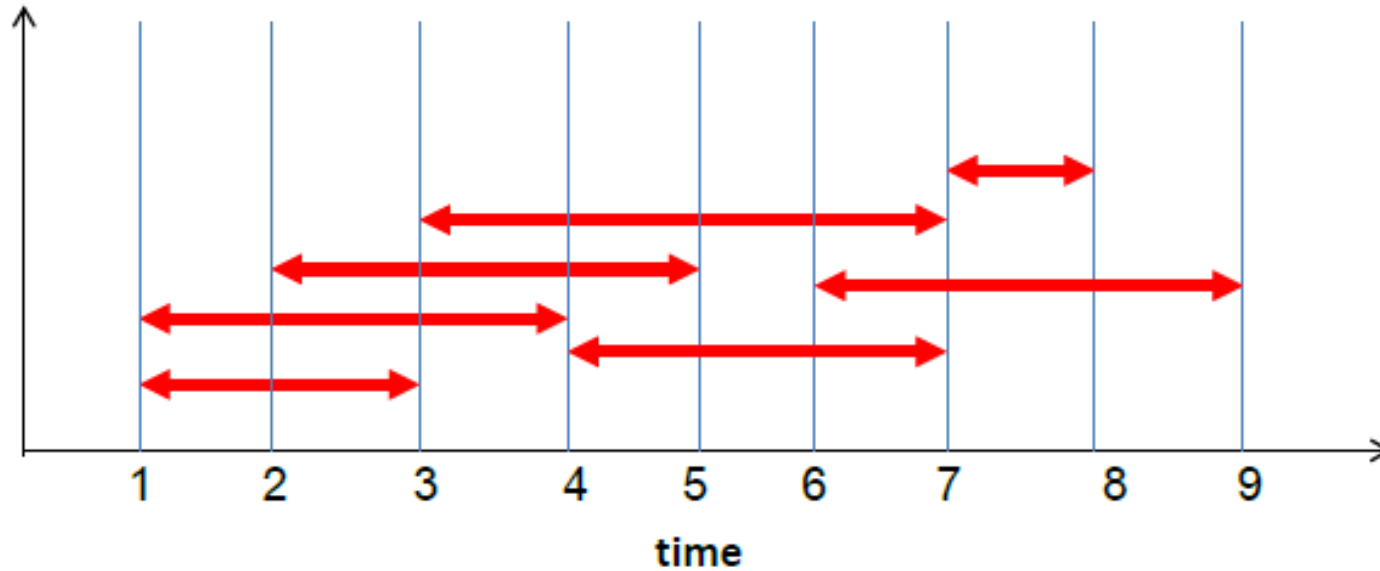
- กำหนดมีเซตของงานทั้งสิ้น n งาน ซึ่งรอที่จะถูกประมวลผลโดย machine แต่ละงานจะประกอบไปด้วย (s_i, f_i) ซึ่งหมายถึงเวลาเริ่มต้นและเวลาสิ้นสุดของการประมวลผลงานที่ i เนื่องจาก machine สามารถประมวลผลได้เพียงครั้งละ 1 งาน ดังนั้นหาก 2 งานมีเวลาซ้อนทับกัน (overlap time) machine จะต้องเลือก 1 งานมาประมวลผลและจะต้องยกเลิกอีกงานไป



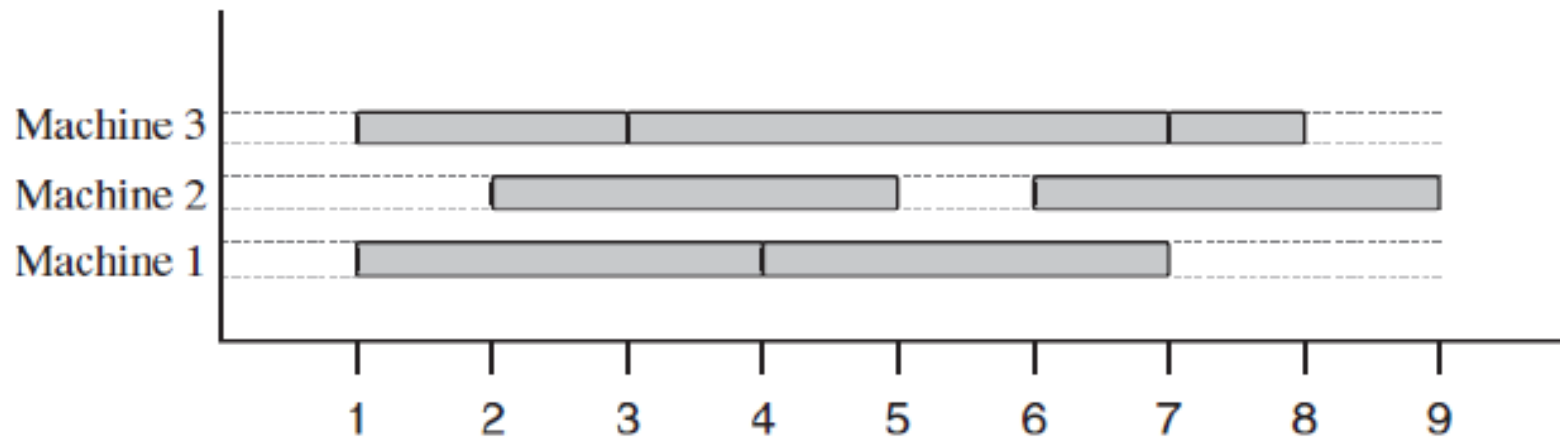
ประเด็นคือต้องการหาจำนวน machine ที่น้อยที่สุดเพื่อรองรับการประมวลผลงานทั้งหมด

หากเลือกประมวลผลงาน 2 จะต้องยกเลิกงาน 1 และงาน 3 หรือต้องการ machine เพิ่มอีก 1 ตัว เพื่อประมวลผลงาน 1 และงาน 3 ตามลำดับ

ตัวอย่าง



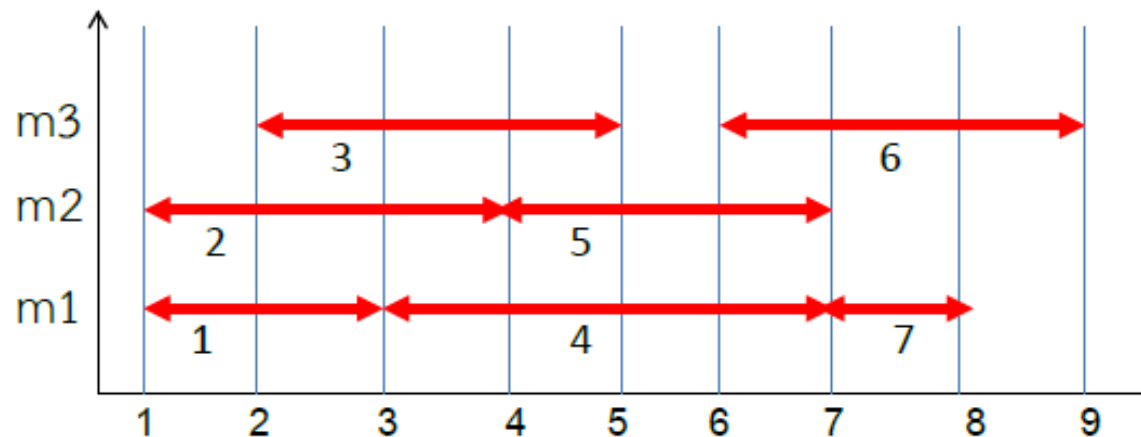
จะต้องใช้ machine อย่างน้อยที่สุด
3 ตัวในการประมวลผลงานทั้งหมด
โดยไม่ขัดแย้ง



Longest-first strategy

เลือกงานที่ใช้เวลาในการประมวลผลนานที่สุดก่อน

- เรียงลำดับงานตามเวลาที่ใช้ในการประมวลผลจากมากไปน้อย
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



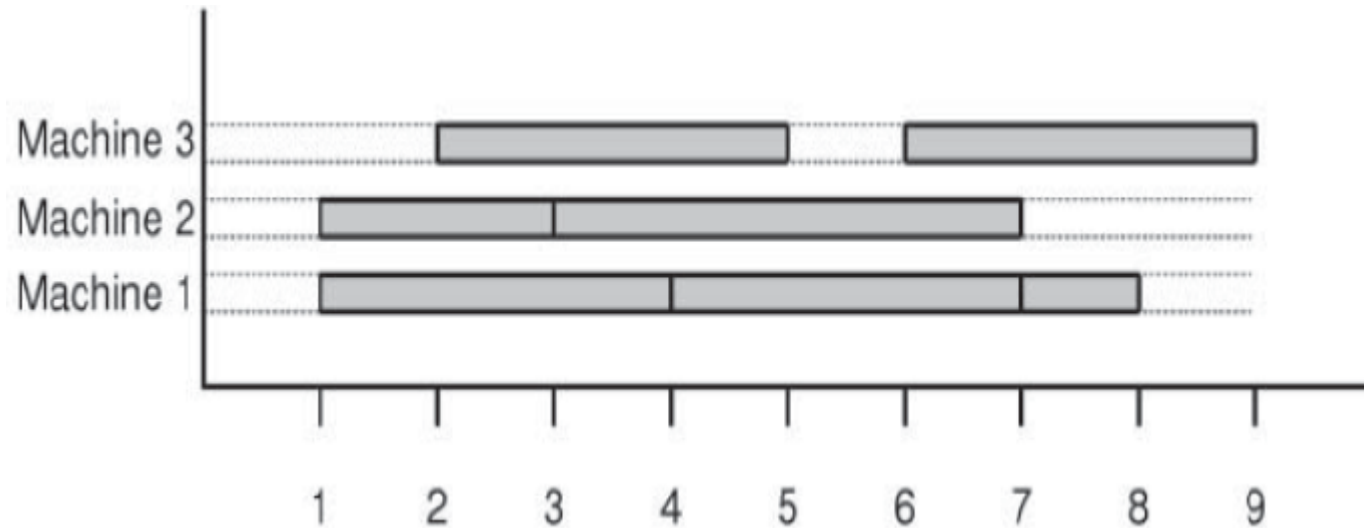
จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	6
2	1	4	2
3	2	5	3
4	3	7	1
5	4	7	4
6	6	9	5
7	7	8	7

Earliest start-time strategy

เลือกงานที่มีเวลาเริ่มต้น (s_i) ก่อน

- เรียงลำดับงานตามเวลาเริ่มต้นจากน้อยไปมาก
- หากเกิดการทับซ้อนเวลาก็เพิ่ม machine อีก 1 ตัว



จะต้องใช้อย่างน้อย 3 machine

Task	s_i	f_i	rank
1	1	3	1
2	1	4	2
3	2	5	3
4	3	7	4
5	4	7	5
6	6	9	6
7	7	8	7

Other strategies for job scheduling

- Earliest finish-time strategy
เลือกงานที่มีเวลาสิ้นสุดก่อน
- Smallest interval strategy
เลือกงานที่ใช้เวลาประมวลผลน้อยที่สุดก่อน
- Least conflict strategy
เลือกงานที่มีช่วงเวลา conflict กับงานอื่นน้อยที่สุดก่อน 26

ตัวอย่างGreedyกับการzipไฟล์

Data Types in C

Type	Size (bits)	Size (bytes)	Range
char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int	16	2	-2^{15} to $2^{15}-1$
unsigned int	16	2	0 to $2^{16}-1$
short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int	32	4	-2^{31} to $2^{31}-1$
unsigned long int	32	4	0 to $2^{32}-1$
float	32	4	3.4E-38 to 3.4E+38
double	64	8	1.7E-308 to 1.7E+308
long double	80	10	3.4E-4932 to 1.1E+4932

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Text Compression

ปกติแล้วคอมพิวเตอร์จัดเก็บไฟล์ในลักษณะที่เป็น fixed-length binary code ตัวอย่างเช่นสมมติไฟล์หนึ่งประกอบไปด้วยตัวอักษรและความถี่ที่ใช้งานดังนี้

Character	Code	Frequency
a	0110 0001	10
e	0110 0101	15
i	0110 1001	12
s	0111 0011	3
t	0111 0100	4
r	0111 0010	1

TOTAL $45 * 8 = 360$ bits

Fixed length binary tree

จะเห็นได้ว่าค่อนข้างจะสิ้นเปลืองจำนวนบิตในการจัดเก็บมาก เนื่องจากมีตัวอักษรที่แตกต่างในไฟล์เพียง 6 ตัวเท่านั้น จึงเป็นไปได้ว่า เราสามารถลดจำนวนบิตลงให้สัมพันธ์กับความแตกต่างของตัวอักษรในไฟล์ได้

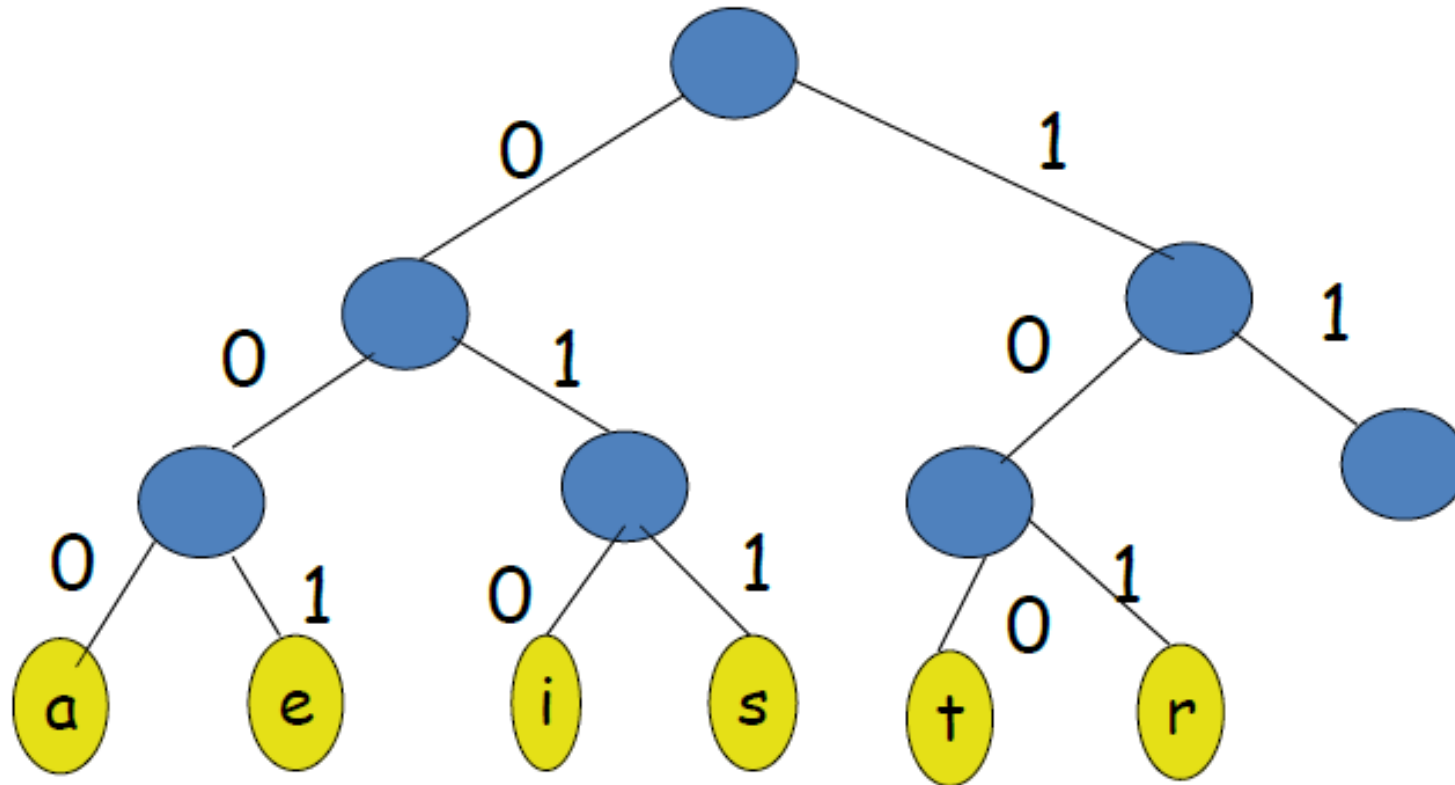
$$n \leq 2^{bit}$$

Character	Code	Frequency
a	000	10
e	001	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL $45 * 3 = 135$ bits

Fixed length binary tree

เราสามารถแสดงต้นไม้ของรหัสไบนารี เพื่อความรวดเร็วสำหรับการถอดรหัสดังนี้



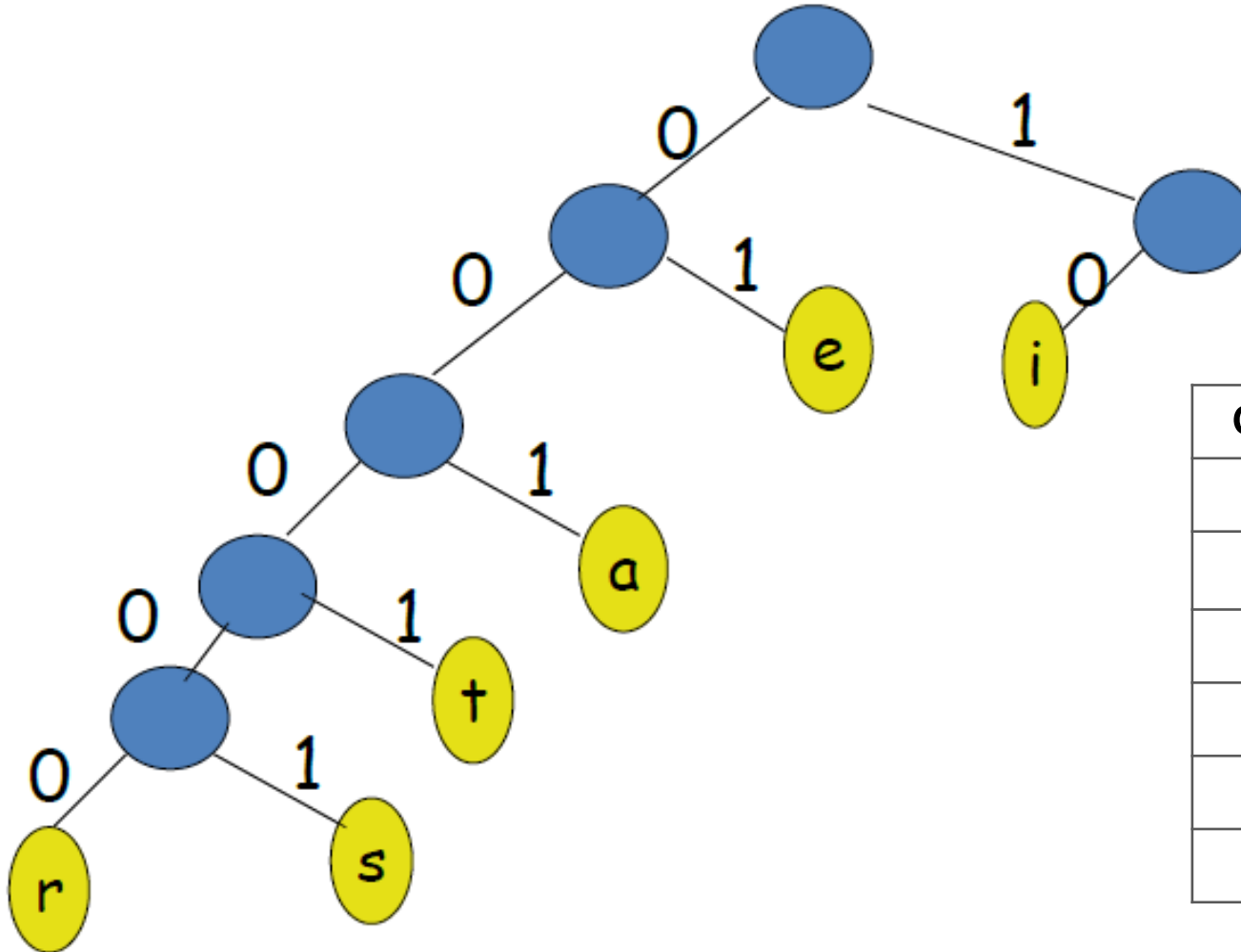
Variable-length binary coding

อย่างไรก็ตามถึงแม้ว่าจะลดพื้นที่ในการจัดเก็บได้ แต่ก็ยังคงไม่ optimal อยู่ดี ลองพิจารณาตัวอย่างด้านล่าง

Character	Code	Frequency
a	000	10
e	11	15
i	010	12
s	011	3
t	100	4
r	101	1

TOTAL $(30 \times 3) + (15 \times 2) = 90 + 30 = 120$ bits

Variable-length binary coding



Character	Code	Frequency
a	001	10
e	01	15
i	10	12
s	00001	3
t	0001	4
r	00000	1

TOTAL = 112 bits

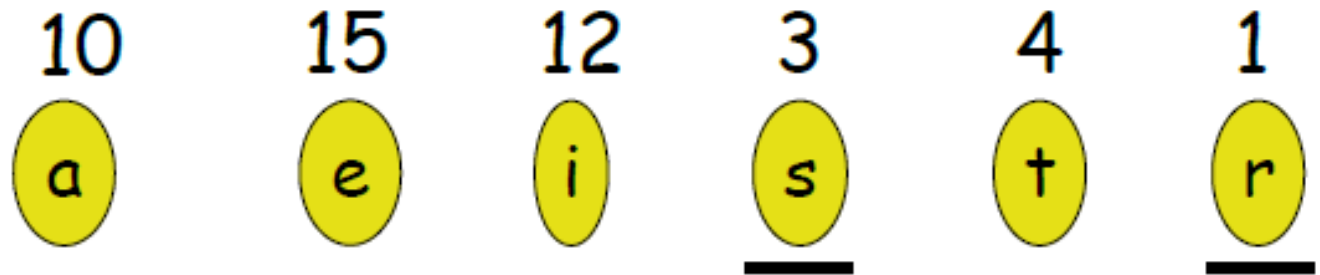
Huffman Coding

Letter ↕	Relative frequency in the English language ↕	
a	8.167%	
b	1.492%	
c	2.782%	
d	4.253%	
e	12.702%	
f	2.228%	
g	2.015%	
h	6.094%	
i	6.966%	
j	0.153%	
k	0.772%	
l	4.025%	
m	2.406%	
n	6.749%	
o	7.507%	
p	1.929%	
q	0.095%	
r	5.987%	
s	6.327%	
t	9.056%	
u	2.758%	
v	0.978%	
w	2.360%	
x	0.150%	
y	1.974%	
z	0.074%	

1. เริ่มจากนำทุกตัวอักษรและความถี่หรือความน่าจะเป็น(probability) ของแต่ละอักษรมาสร้างเป็นไหนด
2. หยิบไหนด 2 ตัวที่มีความถี่ต่ำสุดมาเชื่อมกัน โดยสร้างเป็นต้นไม้หลัก
3. นำไหนดที่มีความถี่น้อยสุดจากไหนดที่เหลือมาเชื่อมกับต้นไม้ย่อยดังกล่าว แต่หากไหนดที่เหลือมีค่าความถี่น้อยกว่าต้นไม้หลัก ให้รวมกับไหนดอื่นเพื่อสร้างต้นไม้ย่อย จากนั้นจึงรวมต้นไม้ย่อยและต้นไม้หลักเข้าด้วยกัน
4. ทำเช่นนี้ไปเรื่อย ๆ จนกระทั่งต้นไม้ย่อยทั้งหมดรวมเป็นต้นไม้ต้นเดียว

Example: Huffman Coding

Character	Frequency
a	10
e	15
i	12
s	3
t	4
r	1



ตัวอักษร s และ r มีความถี่น้อยที่สุด

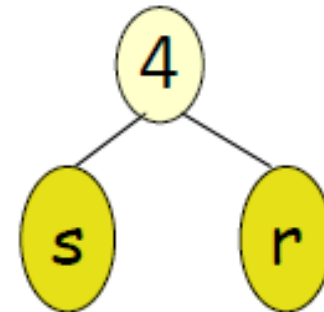
Example: Huffman Coding

10
a

15
e

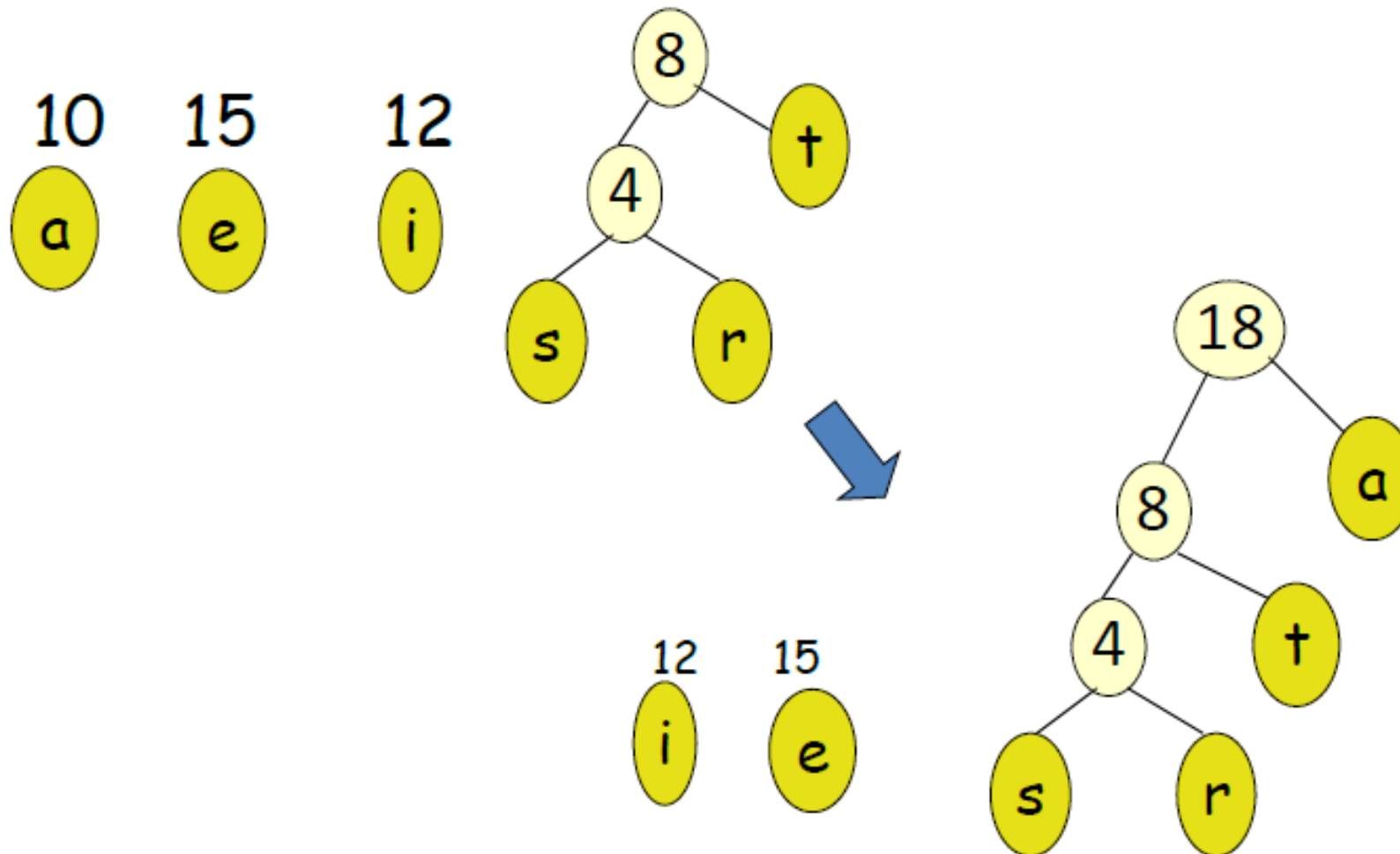
12
i

4
t



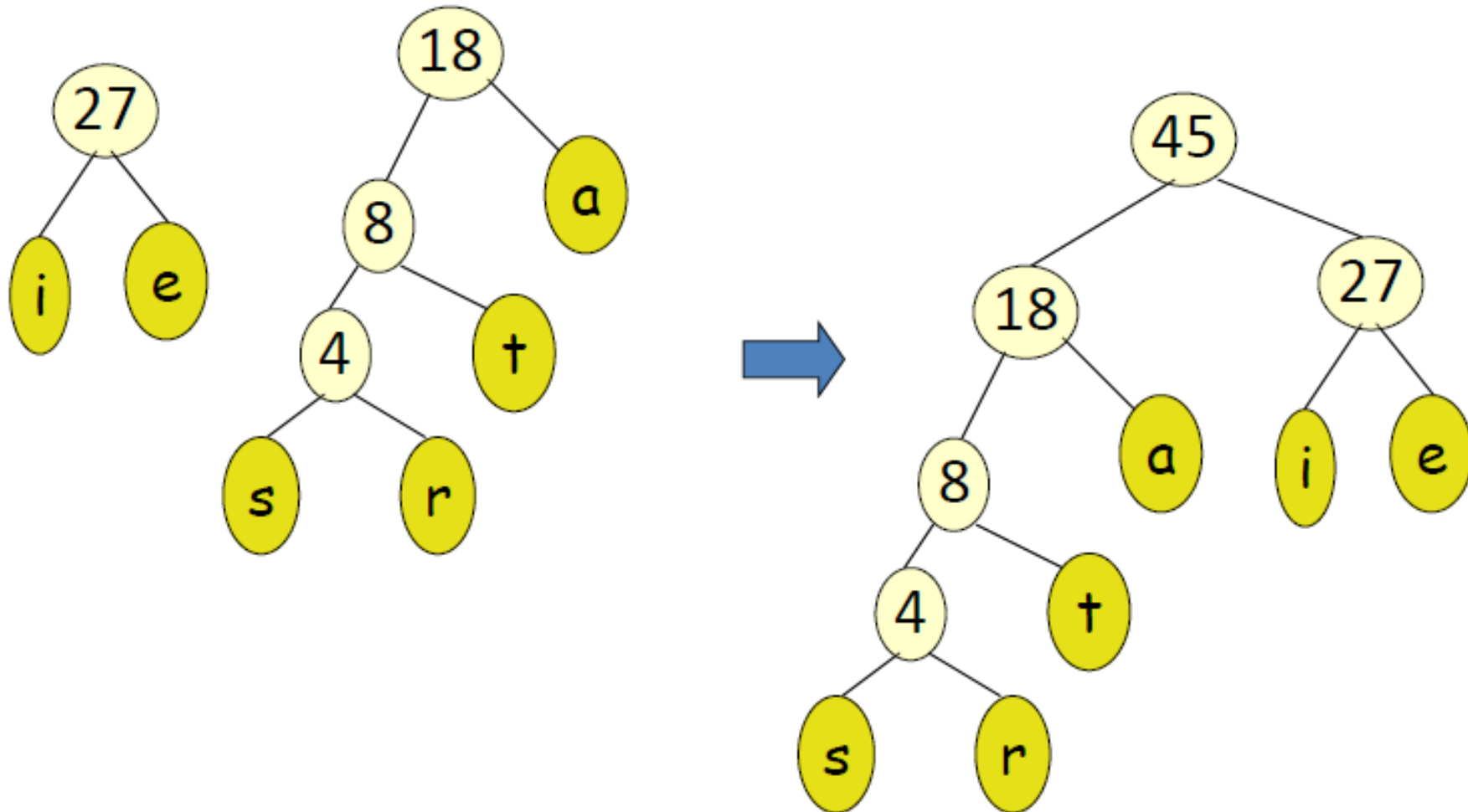
โหนดที่จะนำมารวมกับต้นไม้หลักต้องมีความถี่เท่ากันหรือมากกว่าต้นไม้หลัก

Example: Huffman Coding



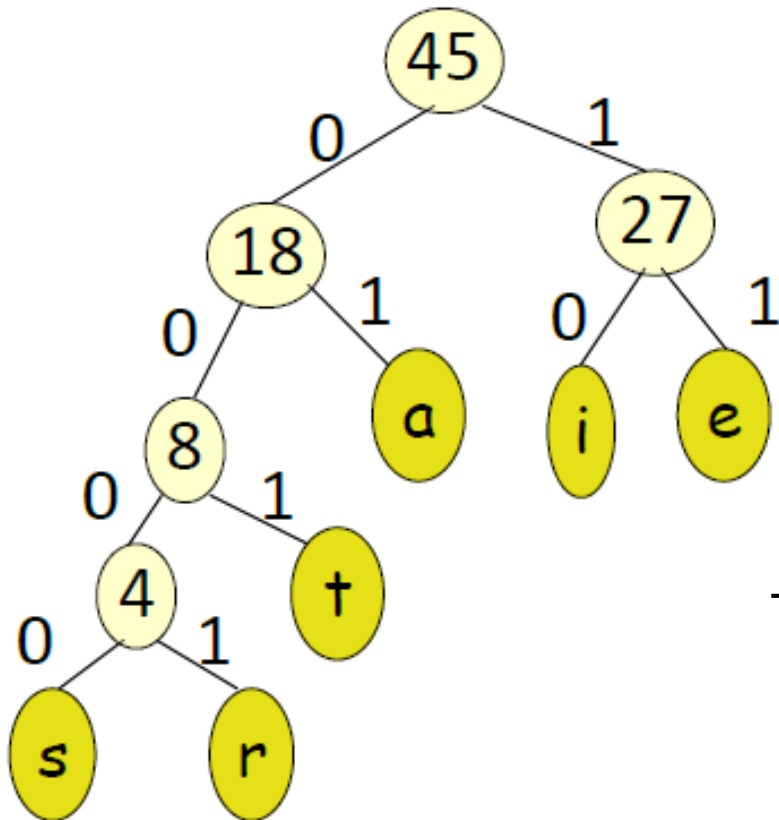
โหนดที่จะนำมารวมกับต้นไม้หลักต้องมีความถี่เท่ากันหรือมากกว่าต้นไม้หลัก

Example: Huffman Coding



Example: Huffman Coding

ให้กิ่งซ้ายมีค่าเป็น 0 และกิ่งขวามีค่าเป็น 1 จากนั้นรหัสของแต่ละตัวอักษรจะเริ่มไล่จาก root node



Character	Code	Frequency
a	01	10
e	11	15
i	10	12
s	0000	3
t	001	4
r	0001	1

$$\begin{aligned}
 \text{TOTAL} &= (10 \times 2) + (2 \times 15) + (2 \times 12) + (4 \times 3) + (3 \times 4) + (4 \times 1) \\
 &= 20 + 30 + 24 + 12 + 12 + 4 \\
 &= \mathbf{102 \text{ bits}}
 \end{aligned}$$

Huffman coding in C

- โค้ดภาษาซีของ Huffman coding <http://www.programminglogic.com/implementing-huffman-coding-in-c/>
- ถูกนำไปใช้งานจริงในแอปพลิเคชันสำหรับการบีบอัดไฟล์ PKZIP และการเข้ารหัสไฟล์ multimedia MP3